

DATA STRUCTURES PROJECT based on complex computing problem

Section A, B

Submission Deadline: 28th November, 2025

Instructions (Must Read)

- **Original Work Only:** Plagiarism or code copying (even partially) will result in a zero. All code will be checked using plagiarism detection tools.
- **Language Requirement:** The project must be implemented in C++ unless stated otherwise. Use proper syntax, naming conventions, and formatting.
- **Code Documentation:**
 1. Add meaningful **comments** explaining important parts of the code.
 2. Use self-explanatory variable and function names.
 3. Each file should include your **name**, **roll number**, and **project title** in comments at the top.
- **Modular Design:** Your project should be logically structured using **classes**, **functions**, and **header files** where appropriate. Avoid writing everything in the main function.
- **Members:** This is a group project, and the maximum number of students allowed in a group is 2.
- **Report Submission:**

Include a brief **report (PDF)** containing:

 - o Explanation of the data structures used
 - o Challenges faced
 - o Workflow Diagram
 - o 5-8 pages length
 - o Work distribution
- Report should be **well-formatted and concise**.
- **Deadline:** Submit the project by the given deadline. Late submissions may incur a penalty.
- **Presentation/Demo:**

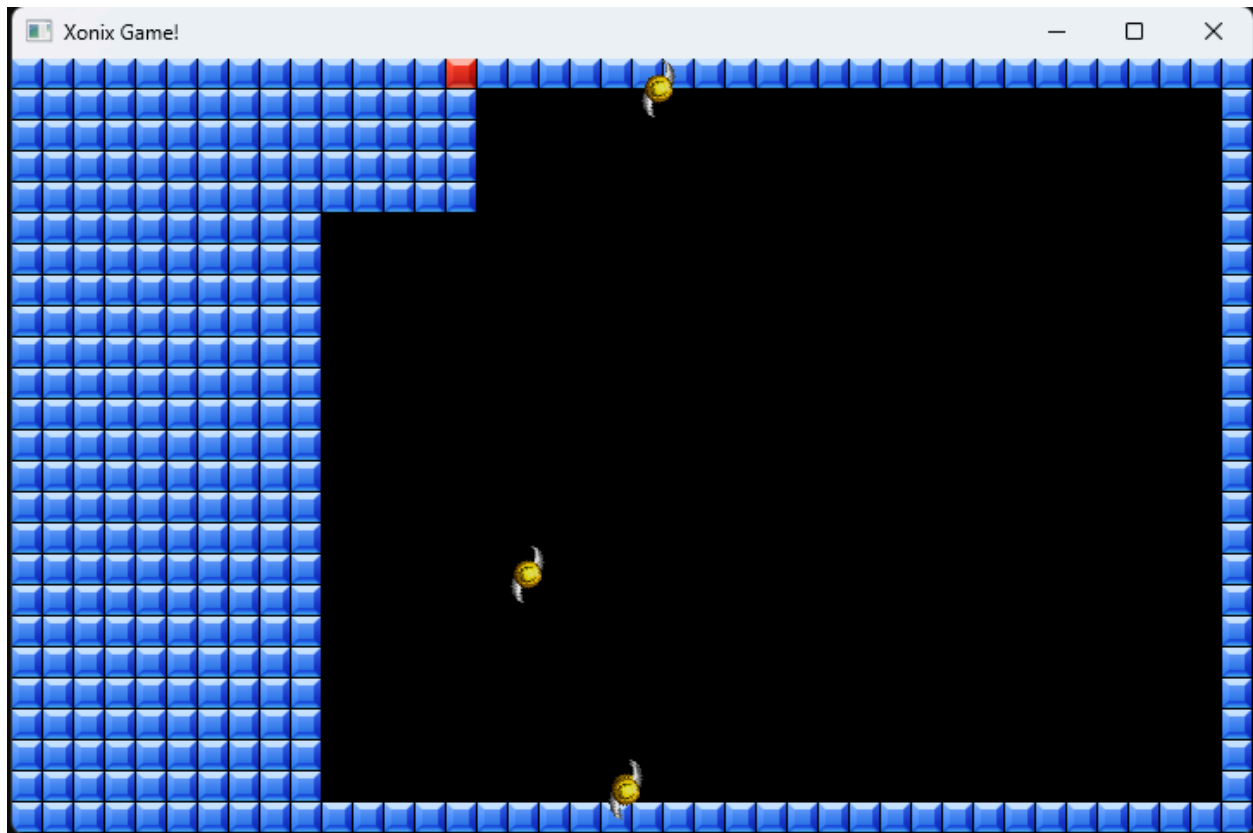
The demo will be carried out from (1 - 3 December, 2025)
- **Submission Format:**

Submit a **ZIP folder** containing:

- o All source code files
 - o Executable (if applicable)
 - o PDF report
 - o Any additional resources (if required)
- **Key Point:**
All the description given must be followed as it is. All the things that are not mentioned in the description are left for you to decide based on your creativity and understanding. This will be judged during your demo. Whatever creative decisions are made, you must be able to justify them during your demo.

Project Description:

You have been given a game, and your task is to implement the additional functionalities. The game given to you is very basic running version and you must include gameplay features as well as a backend system. The game looks as following:



All the basic code and other files related to this game have been provided to you. Understand and then add the required features.

Features to be Implemented:

Main Menu:

Single and Multiplayer Modes:

Start Menu:

- Start Game
- Select Level
- Leaderboard

End Menu:

- End the game properly
- Show final score (highlight if it's a new high score)

- Options: Restart, Main Menu, Exit Game

Apart from these features use your creative sense to add necessary options in the menu based on the implementation of various features mentioned next.

Login and Authentication:

To simulate a real-world gaming system, the project must implement a **Login and Registration module** with proper validation and data persistence.

Key Requirements:

Player Registration:

- New players should be able to register by providing:
 - A unique username
 - A secure password
 - Optional additional details (e.g., nickname, email, etc.)
- Perform input validation:
 - Ensure the username is not already taken.
 - Check password strength (e.g., minimum length).
- After validation, **store the player's credentials in a file** using file handling.
 - You may store additional info like registration timestamp or a unique player ID.
 - Keep the format structured

Player Login:

- Players should be able to log in using their **username and password**.
- On login:
 - Retrieve and search the credentials file.
 - Validate the username exists.
 - Verify that the entered password matches the stored password.
- Only after **successful authentication**, allow access to player features like Profile, Matchmaking, and Game Room.

Points System:

- Capturing a tile grants 1 point.
- Capturing more than 10 tiles in a single move awards double (×2) points.
- A reward counter tracks how many times this bonus has been earned:
 - After 3 occurrences, the threshold reduces to 5 tiles for the double points bonus.
 - After 5 total occurrences, capturing more than 5 tiles yields ×4 points.
- Upon reaching a score of 50, the player earns a power-up that stops all enemies for 3 seconds.
 - The power-up can be used any time until the end of the game.
 - Additional power-ups are granted at scores of 70, 100, 130, and for every additional 30 points.
 - Unused power-ups stack and are stored in the player's inventory.

Player Profile:

Each player has a unique profile showing:

- Username
- Friends list
- Match history (win/loss)
- Total points

Multiplayer mode:

- All features apply to both players.
- Both players play on the same game board:
 - Player 1 Controls - Arrow keys
 - Player 2 Controls - "Q", "G", "A", "X"
- Shared game timer
- Individual scores and power-ups are displayed on their respective sides of the screen.

- Since both players share the same game board, they can interact with each other. If one player touches the other player or their in-progress (constructing) tiles, the following rules apply:
 - o If both players collide while they are moving (constructing tiles), both players will die.
 - o If Player 1 touches the constructing tile of Player 2, Player 1 will die – and vice versa.
 - o If Player 1 collides with Player 2 while Player 1 is constructing tiles and Player 2 is not, Player 1 will die – and vice versa.
- Power-up usage in two-player mode:
 - o Stops both the enemies and the other player for 3 seconds.
- When both players are eliminated, the game ends and the winner is determined based on the highest score.

Leaderboard:

The system must maintain a **Leaderboard** that displays the **Top 10 players** based on their total points. This leaderboard should **automatically update** whenever a match is played, and scores change.

Key Requirements:

- **Heap Data Structure:**
 - o Implement the leaderboard using a **Min-Heap** of fixed size 10.
 - o This will ensure that only the top 10 scoring players are stored at any time.
 - o The root of the min-heap will always hold the player with the **lowest score** among the top 10.
 - o When a player with a score **higher than the root** enters or updates their score, the root is replaced, and the heap is restructured.
- **Updating Leaderboard:**
 - o After each match, check if either player should enter or affect the leaderboard.
 - o If the leaderboard has fewer than 10 players, insert them directly.
 - o If the heap is full and the new player's score is higher than the current minimum, replace the root and heapify.

- **Display Format:**
 - o Sort the heap in descending order (temporarily for display purposes) to show rankings from highest to lowest.
 - o Display Player ID and Total Score for each entry.
- **Data Handling:**
 - o Ensure that the player scores are synced between the Player Profile and the heap to avoid inconsistencies.
 - o You may store a reference or index to the player in the heap nodes to avoid redundancy.

Match Making Queue:

To simulate a competitive multiplayer environment, the project should implement a **priority queue matchmaking system**. This queue is responsible for pairing players into matches based on their performance—specifically their **total points or scores**.

Key Requirements:

- **Use of a Priority Queue:**
 - o Players waiting for a match should be inserted into a priority queue.
 - o The **priority is determined by the player's total score** (higher score = higher priority).
 - o The player with the highest score should be matched first.
- **Matchmaking Logic:**
 - o As players enter the queue, the system should check if there is at least one other player available to pair.
 - o The two players with the **highest priorities (scores)** should be dequeued and matched.
 - o After the match is completed, their scores should be updated and the queue should continue matching others as long as there are enough players.

Game Room:

- Use a queue data structure to handle players waiting for a match.
- Automatically pair players from the queue into a game room.

- You must implement priority queue here and player priority is determined by the leaderboard.

Send/Accept Friend Requests:

Players can build social connections by sending and accepting friend requests. This system should replicate how real-world social features work using fundamental data structures.

- **Array List of Players:**
 - o Maintain an array list (dynamic array) where each index stores a player object.
 - o Each player object contains:
 - Player ID (username or unique identifier)
 - Linked list of friends (Player IDs)
- **Linked List of Friends (Per Player):**
 - o For each player in the array list, maintain a linked list that stores all their friends.
 - o This list gets updated as players send and accept requests.
- **Hash Table for Fast Search:**
 - o Implement a hash table that maps Player IDs (usernames) to their index in the array list.
 - o This enables fast searching when:
 - Sending friend requests
 - Accepting/rejecting requests
 - Looking up mutual friends or shared connections

Functional Requirements:

- **Send Friend Request:**
 - o A player can search for another player using their username.
 - o The hash table helps locate the target player quickly.
 - o If the request is valid (not already friends, not pending), store the request.
- **Accept/Reject Friend Request:**
 - o Pending requests can be accepted or rejected.

- o If accepted, both players should be added to each other's linked list of friends.
- **View Friends List:**
 - o Players can view their current list of friends by traversing their linked list.
- **Search and Add Functionality:**
 - o Hash table ensures constant-time lookup for players.
 - o Use simple hash functions or maps to index by Player ID.

Save Game:

Players should be able to **save their in-progress games** at any point and resume later from exactly where they left off. To achieve this:

- When a player starts a game, a **GameState structure** (or class) must be initialized. This structure should include:
 - o A **timestamp** of the save.
 - o The **Player ID** (or username).
 - o A **linked list** of tiles (or game elements) the player has created or interacted with so far.
- This structure must be **serialized and saved to a file**, with a **unique Save ID** (a string identifier) associated with it.
- The player should later be able to **resume** the game by entering this Save ID, and the system should:
 - o Retrieve the corresponding saved file.
 - o Reconstruct the game state from the file.
 - o Continue the game from the exact saved point.

This feature will demonstrate your ability to:

- Build and manipulate custom data structures (e.g., linked lists).
- Implement file I/O for structured data.
- Serialize and deserialize class/struct objects.
- Handle dynamic memory effectively.

Inventory:

To enhance the user experience and implement advanced data structure usage, the project should include a feature where players can **browse and select visual themes** from an inventory.

Core Concept:

The **Inventory System** stores a collection of themes (backgrounds, colors, tile styles, etc.), each identified by a unique **Theme ID**. These themes are organized in an **AVL Tree** to allow **efficient searching, insertion, and retrieval**.

Key Requirements:

Initialization:

- At the start of the game, a predefined set of themes should be **loaded into an AVL Tree**.
- Each theme should include:
 - A **Theme ID** (used as the key)
 - A **Theme Name**
 - A **short description**
 - Optional visual preview or simulated color code (in text mode)

AVL Tree Functionality:

- Implement an **AVL Tree** (self-balancing Binary Search Tree).
- Ensure that:
 - Tree remains balanced after each insertion.
 - **Search, insertion, and traversal** operations are efficient (logarithmic time complexity).
- Provide operations to:
 - **Search** for a theme by ID or name
 - **Display all themes** in sorted order by ID
 - Optionally support deletion or updates of themes

Theme Selection:

- Allow the player to **browse themes** via in-order traversal (sorted order).
- Allow the player to **search for a theme by ID** to quickly find and apply it.
- Once selected, the player's theme preference can be saved to their profile using file handling.

Project Report:

As this is a **group-based project (maximum 2 members)**, each group must submit a comprehensive **project report** along with their code and demonstration files.

The report should document the entire workflow, distribution of tasks, implementation challenges, and learning outcomes. This ensures accountability, clarity of effort, and professional documentation skills.

Required Sections of the Report:

1. Cover Page

- o Project Title
- o Group Number
- o Full names and roll numbers of both members
- o Course Name and Section
- o Instructor Name
- o Submission Date

2. Introduction

- o A brief overview of the game/project.
- o Key goals and features implemented.
- o Tools/languages used.

3. Work Distribution

- o A clear table showing how the work was divided between the two members.
- o Include feature-wise or module-wise distribution.

4. Workflow s Implementation Timeline

- o Describe the planning and development stages.
- o Optional: Include a Gantt chart or a list of development milestones.

5. Data Structures Used

- o Clearly explain the use of each major data structure (e.g., Linked List, Heap, AVL Tree, Hash Table).
- o Justify **why** each structure was chosen for its feature.

6. Challenges Faced s Solutions

- o Discuss 2-3 key challenges during development (technical or teamwork- related).
- o Describe how the group overcame them.

7. Screenshots or Sample Outputs

- o Add relevant images to showcase GUI, menus, console output, leaderboard display, etc.

8. Conclusion

- o What the group learned from the project.
- o Suggestions for improvements or future features.

G. Appendix (Optional)

- o Include any references, pseudocode snippets, or additional notes.

Report Submission Guidelines:

- **Format:** PDF.
- **Length:** 5-8 pages (excluding appendix)
- **Filename:** GroupX_ProjectReport.pdf (replace X with your group number)
- Must be submitted along with the codebase and executable file or instructions to run.

Game Link:

<https://github.com/embeddedmz/16-Games/tree/master/09%20Xonix>

Allowed Libraries:

- <SFML/Graphics.hpp>
- <time.h>
- <Cmath>
- <cstdlib>

- `<string>`
- `<cstring>`

Restricted Libraries and functions:

- `<vector>`
- `<algorithm>`
- Built-in lists
- Built-in queues
- Built-in stack

Evaluation Rubric /Project Feature Checklist (Compact Version)

Feature	Marks	Status (✓/X)	Notes
Main Menu	5		(Show all options clearly)
Login & Registration	10		(Save to file; check duplicates)
Authentication	5		(Validate username & password)
Save Game	10		(Save player object with tiles list)
Friend System	10		(ArrayList + LinkedList + HashTable)
Matchmaking Queue	10		(Priority based on scores)
Leaderboard	10		(Top 10 players via Min-Heap)
Inventory System	10		(Themes stored in AVL Tree)
Multiplayer Mode	10		(Match 2 players, game room)
Player Profile	5		(Points tracking, theme selection)
Report Writing	20		(Cover page, workflow, screenshots)
Code Quality	10		(Classes, functions, clean style)

RESTRICTIONS

You are not allowed to use any built-in library for any data structures. All the data structures used should be built from scratch. Violation will result in serious deduction of marks. The purpose of project is to make you able to build and learn real world applications of data structures studies. It is important to state that in evaluation your efforts and honesty will be of highest precedence and will give you better chance of scoring good score then cheating. You are not allowed to use someone else code as this will be considered plagiarism and will result in 0 marks best case and failing the course worst case. You are not allowed to use AI generated code and if found will result in same action as in case of cheating. The report also must not be generated using AI or copied from someone else as this will also result in same action as mentioned above.

Good Luck with your project.